Università degli studi di Messina

DIPARTIMENTO DI SCIENZE MATEMATICHE E INFORMATICHE, SCIENZE FISICHE E SCIENZE DELLA TERRA
Bachelor's Degree in Computer Science L31
Curriculum: Data Analysis

Comparative Analysis of Database Performance in a Course Management System

Supervisor:
Prof. Armando Ruggeri

Candidate:
Yoseph Endale

Accademic Year:
2022/2023

Contents

1 Introduction **1**
1.1 Problem Statement . . . 1
1.2 Objectives . . . 1
1.3 Thesis Structure . . . 2

2 State of the Art: Database Paradigms, Architectures, and Tooling **3**
2.1 The Relational Model (RDBMS): MySQL . . . 3
2.1.1 Storage Engine: InnoDB and B+ Tree Internals . . . 4
2.1.2 Concurrency and MVCC . . . 4
2.1.3 Limitations and Trade-offs . . . 4
2.1.4 Tooling: phpMyAdmin . . . 4
2.2 The Document Store: MongoDB . . . 5
2.2.1 Data Model: BSON and Flexible Schema . . . 5
2.2.2 Storage Engine and Execution . . . 5
2.2.3 Aggregation Framework . . . 5
2.2.4 Consistency Model and Trade-offs . . . 5
2.2.5 Tooling: MongoDB Compass . . . 6
2.3 Wide-Column Store: Apache Cassandra . . . 6
2.3.1 Architectural Foundations . . . 6
2.3.2 LSM-Tree Storage Engine . . . 6
2.3.3 Trade-offs . . . 7
2.4 Graph Database: Neo4j . . . 7
2.4.1 Data Model: Property Graph . . . 7
2.4.2 Index-Free Adjacency . . . 7
2.4.3 Query Execution . . . 8
2.4.4 Trade-offs . . . 8
2.5 Key-Value Store: Redis . . . 8
2.5.1 Architecture . . . 8
2.5.2 Data Structures . . . 8
2.5.3 Performance Characteristics . . . 9
2.5.4 Trade-offs . . . 9

2.6 Cross-Paradigm Comparison and CAP Theorem Analysis 9
2.7 Research Infrastructure: Docker Containerization 10
2.7.1 Advantages 10
2.7.2 Scientific Impact 10

3 Materials and Methods 11
3.1 System Specification and Infrastructure 11
3.1.1 Hardware Configuration 11
3.1.2 Software Stack 11
3.2 Infrastructure Orchestration via Docker 12
3.3 Synthetic Data Generation: `engine.py` 12
3.3.1 Logical Integrity and Relationship Simulation 12
3.4 Cross-Paradigm Data Modeling 12
3.5 Benchmark Workload Design 13
3.5.1 Q1: Primitive Point Lookup 13
3.5.2 Q2: Attribute Filtering 13
3.5.3 Q3: Multi-Table Join 13
3.5.4 Q4: Multi-Hop Traversal 13
3.6 Measurement and Statistical Methodology 13

4 Results and Discussion 15
4.1 Data Ingestion Performance 15
4.2 Query Execution Performance 16
4.2.1 Mean Execution Metrics 16
4.2.2 First Execution Metrics (Cold Start) 17
4.2.3 Analysis of First Run vs. Average Execution 18
4.2.4 Q1: Simple Point Lookup 18
4.2.5 Q2: Attribute Filtering 19
4.2.6 Q3: Multi-Table Join 20
4.2.7 Q4: Multi-Hop Relationship Traversal 20
4.3 Architectural Trade-offs and CAP Implications 21
4.4 Statistical Reliability and Variability 21

5 Conclusions and Future Works 22

List of Figures

4.1 Data Ingestion Performance: Bulk Loading Time (Log Scale) 15
4.2 Q1 execution time: primary key lookup 19
4.3 Q2 execution time: attribute-based filtering 19
4.4 Q3 execution time: three-way join 20
4.5 Q4 execution time: multi-hop traversal (log scale) 21

List of Tables

3.1 Structural mapping strategies across database paradigms 13

4.1 Total ingestion time (seconds) across dataset scales 16
4.2 Comprehensive Average Query Execution Times (ms) 17
4.3 First Execution Query Times (ms) — Cold Start 18

Abstract

As educational institutions transition toward massive digital infrastructures, the choice of a database management system (DBMS) becomes a critical factor in application performance and scalability. This thesis presents a comprehensive performance analysis of five distinct database paradigms: Relational (MySQL), Document-oriented (MongoDB), Key-Value (Redis), Wide-Column (Cassandra), and Graph (Neo4j), within the context of a Course Management System (CMS). Using a synthetic dataset scaling up to one million records, we evaluate these systems across data ingestion speed and query complexity. Our findings reveal significant trade-offs: while MySQL remains stable for complex joins, Neo4j excels in relationship traversal, and MongoDB offers rapid ingestion at the cost of high-complexity query latency. The research concludes with an architectural decision matrix to guide developers in selecting the appropriate data store based on specific CMS workload characteristics.

Chapter 1

Introduction

The modern university ecosystem generates a vast amount of interconnected data, from student enrollments to departmental resource allocation. Traditionally, Relational Database Management Systems (RDBMS) have been the exclusive choice for such systems. However, the rise of "Big Data" and the need for flexible schemas have popularized the NoSQL movement.

1.1 Problem Statement

In a Course Management System, data is highly relational but also varies in access patterns. A point-lookup for a student profile requires different optimizations than a deep search through a graph of course prerequisites. The central problem addressed in this research is: *Which database paradigm maintains the highest performance efficiency as the dataset scales from a small university (250,000 records) to a large-scale institution (1,000,000 records)?*

1.2 Objectives

The primary objectives of this research are:

1. To deploy a multi-paradigm database environment using **Docker** containerization.
2. To measure the "Bulk Loading" efficiency of each system using standardized CSV inputs.
3. To execute and benchmark four distinct query types (Simple to Highly Complex) across varying scales.

4. To provide a comparative analysis of the administrative tools used, specifically **phpMyAdmin** for MySQL and **MongoDB Compass** for NoSQL visualization.

1.3 Thesis Structure

This thesis is organized as follows: Chapter 2 explores the "State of the Art" and the theoretical mechanics of each tool. Chapter 3 describes the experimental environment and Docker orchestration. Chapter 4 presents the empirical data, followed by a discussion and conclusion in Chapter 5.

Chapter 2

State of the Art: Database Paradigms, Architectures, and Tooling

This chapter provides a comprehensive and critical analysis of five major database paradigms evaluated in this research: Relational (MySQL), Document (MongoDB), Wide-Column (Cassandra), Graph (Neo4j), and Key-Value (Redis). Beyond a descriptive overview, this chapter examines their internal data structures, consistency models, execution strategies, and architectural trade-offs.

A dataset of 1,000,000 records was used to evaluate performance under realistic workloads. Additionally, we analyze the tooling ecosystem and the role of containerization in ensuring experimental reproducibility.

2.1 The Relational Model (RDBMS): MySQL

The relational model, introduced by E.F. Codd, remains the foundation of modern data management systems. MySQL represents this paradigm with strong guarantees of consistency, integrity, and structured query capabilities through SQL.

Relational systems prioritize **ACID properties**:

- **Atomicity**: Transactions are all-or-nothing.
- **Consistency**: Constraints ensure valid states.
- **Isolation**: Concurrent transactions do not interfere.
- **Durability**: Committed data survives system failures.

2.1.1 Storage Engine: InnoDB and B+ Tree Internals

The InnoDB storage engine uses B+ Trees as its fundamental indexing structure.

- **Tree Structure:** Internal nodes store keys and pointers, while all actual data resides in leaf nodes. Leaf nodes are linked sequentially, enabling efficient range scans.
- **Fan-out and Disk Optimization:** Each node aligns with disk page sizes (typically 16KB), maximizing fan-out and minimizing tree depth. Even large datasets require only a few I/O operations for lookup.
- **Clustered Indexing:** Data is physically stored in primary key order. Secondary indexes store references to the primary key, introducing an additional lookup step.

2.1.2 Concurrency and MVCC

InnoDB implements **Multi-Version Concurrency Control (MVCC)**, allowing non-blocking reads:

- Readers access snapshot versions of data.
- Writers create new versions instead of overwriting.
- Undo logs maintain previous states for rollback and isolation.

2.1.3 Limitations and Trade-offs

Despite strong consistency, relational systems exhibit:

- Expensive JOIN operations at scale.
- Limited horizontal scalability.
- Rigid schema evolution.

2.1.4 Tooling: phpMyAdmin

phpMyAdmin provided:

- Schema visualization and normalization validation.
- Index inspection to prevent full table scans.
- Enforcement verification of referential integrity constraints.

2.2 The Document Store: MongoDB

MongoDB represents a schema-flexible, document-oriented paradigm optimized for scalability and developer agility.

2.2.1 Data Model: BSON and Flexible Schema

MongoDB stores data in BSON format, extending JSON with binary encoding.

- **Schema Flexibility:** Documents within a collection may differ structurally.
- **Embedding:** Related data can be nested, reducing the need for joins.
- **Referencing:** Alternatively, relationships can be maintained via references, similar to foreign keys.

2.2.2 Storage Engine and Execution

MongoDB uses the WiredTiger storage engine:

- Document-level locking improves concurrency.
- Compression reduces storage footprint.
- Indexes are implemented as B-Trees (not B+ Trees).

2.2.3 Aggregation Framework

Complex queries are executed via aggregation pipelines:

- Sequential stages transform data.
- `$lookup` simulates join operations.
- Performance depends heavily on pipeline ordering and indexing.

2.2.4 Consistency Model and Trade-offs

MongoDB offers tunable consistency:

- Default: eventual consistency in distributed setups.
- Optional strong consistency via write concerns.

Trade-offs include:

- Data duplication when embedding.
- Increased application-level responsibility.
- Document growth limitations.

2.2.5 Tooling: MongoDB Compass

Compass enabled:

- Visual query construction.
- Execution plan analysis (COLLSCAN vs IXSCAN).
- Aggregation pipeline debugging.

2.3 Wide-Column Store: Apache Cassandra

Cassandra is a distributed, highly available database designed for write-heavy workloads and large-scale systems.

2.3.1 Architectural Foundations

Cassandra follows an **AP model** under the CAP theorem:

- Prioritizes Availability and Partition Tolerance.
- Consistency is tunable per query.

2.3.2 LSM-Tree Storage Engine

Cassandra uses Log-Structured Merge Trees:

- **Write Path:**
  - Writes are appended to a commit log.
  - Stored in Memtables (in-memory).
  - Flushed to disk as immutable SSTables.
- **Read Path:**
  - Queries must check multiple SSTables.
  - Bloom filters reduce unnecessary disk reads.

- **Compaction:**
  - Periodic merging of SSTables.
  - Causes I/O amplification and latency spikes.

2.3.3 Trade-offs

Advantages:

- Extremely fast writes.
- Linear horizontal scalability.

Disadvantages:

- Complex data modeling (query-driven design).
- Expensive reads.
- Compaction overhead.

2.4 Graph Database: Neo4j

Neo4j is designed for highly connected data, where relationships are central.

2.4.1 Data Model: Property Graph

- Nodes represent entities.
- Relationships are first-class citizens.
- Both nodes and relationships can store properties.

2.4.2 Index-Free Adjacency

- Each node directly references its neighbors.
- Traversals are pointer-based, not index-based.
- Complexity depends on traversal depth, not dataset size.

2.4.3 Query Execution

Neo4j uses the Cypher query language:

- Pattern-matching paradigm.
- Optimized for multi-hop queries.

2.4.4 Trade-offs

Advantages:

- Superior performance for relationship-heavy queries.
- Eliminates costly JOIN operations.

Disadvantages:

- Not optimized for bulk data aggregation.
- Memory-intensive for large graphs.

2.5 Key-Value Store: Redis

Redis is an in-memory key-value store optimized for ultra-low latency.

2.5.1 Architecture

- Single-threaded event loop (no locking overhead).
- Data stored entirely in RAM.
- Supports persistence via RDB snapshots and AOF logs.

2.5.2 Data Structures

Redis supports:

- Strings
- Hashes
- Lists
- Sets and Sorted Sets

2.5.3 Performance Characteristics

- Microsecond-level latency.
- Ideal for caching and real-time systems.

2.5.4 Trade-offs

- Limited by available memory.
- Not suitable for complex queries.
- Persistence introduces trade-offs between durability and performance.

2.6 Cross-Paradigm Comparison and CAP Theorem Analysis

Each database reflects a different trade-off space:

- **MySQL:** CA (Consistency + Availability)
- **MongoDB:** CP/AP (configurable)
- **Cassandra:** AP (Availability + Partition Tolerance)
- **Neo4j:** CA (optimized for traversal)
- **Redis:** CA (in-memory, optional persistence)

Workload Suitability:

- Read-heavy: MySQL, Redis
- Write-heavy: Cassandra
- Flexible schema: MongoDB
- Relationship-heavy: Neo4j

No single system dominates; performance depends on workload characteristics and consistency requirements.

2.7 Research Infrastructure: Docker Containerization

To ensure experimental rigor, all databases were deployed using Docker containers.

2.7.1 Advantages

- **Isolation:** Prevents resource interference.
- **Reproducibility:** Identical environments across experiments.
- **Stateless Testing:** Containers reset between runs to eliminate caching bias.

2.7.2 Scientific Impact

Containerization ensures that performance differences are attributable solely to database architectures rather than environmental inconsistencies, thereby strengthening the validity of the experimental results.

Chapter 3

Materials and Methods

This chapter details the experimental framework designed to evaluate the performance of diverse database paradigms under varying workloads. To ensure high research integrity and technical rigor, the experimental environment was constructed upon three architectural pillars: **Isolated Containerization** for environment parity, **Relationally Consistent Synthetic Data Generation** for logical coherence at scale, and **Multi-Iterative Statistical Benchmarking**.

3.1 System Specification and Infrastructure

All benchmarks were conducted on a dedicated workstation to minimize non-deterministic variables such as background operating system processes, thermal throttling, and network latency.

3.1.1 Hardware Configuration

- **Central Processing Unit (CPU):** [e.g., AMD Ryzen 7 5825U] (8 cores, 16 threads).
- **Random Access Memory (RAM):** [e.g., 8 GB DDR4 3200 MHz].
- **Storage:** [e.g., 500 GB NVMe M.2 SSD], selected to minimize I/O bottlenecks during bulk data loading.

3.1.2 Software Stack

- **Host Operating System:** [e.g., Ubuntu 24.04.4 LTS].
- **Containerization Platform:** Docker Engine v24.0.7 with Docker Compose v2.21.0.

- **Benchmarking Language:** Python 3.12.1.
- **Database Drivers:** `mysql-connector-python` (v8.2), `pymongo` (v4.6), `neo4j` (v5.14), `redis` (v5.0).

3.2 Infrastructure Orchestration via Docker

With the exception of the host-resident MySQL instance, all NoSQL databases were deployed as independent services using **Docker**. This containerized approach ensures reproducibility and isolation.

1. **Deterministic Resource Allocation:** Each container was constrained using Docker resource limits (e.g., 4 GB RAM and fixed CPU shares). This enables controlled observation of how different storage engines (e.g., LSM-Trees vs. B+ Trees) behave under memory pressure at large data volumes.
2. **Stateless Execution Protocol:** A strict clean-state policy was enforced. Between each dataset size (250K $\rightarrow$ 500K $\rightarrow$ 750K $\rightarrow$ 1M records), containers were stopped, associated volumes deleted, and services redeployed. This eliminates residual caching effects and ensures experimental consistency.

3.3 Synthetic Data Generation: `engine.py`

A custom data generation framework, `engine.py`, was implemented using the **Faker** library to populate a Course Management System (CMS) dataset with realistic records.

3.3.1 Logical Integrity and Relationship Simulation

The generator maintains referential consistency across entities:

- **Entity Mapping:** During generation of `Enrollment` records, valid `student_id` and `section_id` values are sampled from pre-generated entity pools.
- **Controlled Distribution:** Enrollment counts follow realistic distributions (e.g., 3–6 enrollments per student), ensuring that query complexity reflects real-world workloads rather than uniform randomness.

3.4 Cross-Paradigm Data Modeling

A central challenge in this research was transforming a single logical dataset into multiple physical representations across database paradigms.

Paradigm	Storage Format	Optimization Strategy
Relational	Normalized tables (3NF)	B+ Tree primary and foreign key indexes
Document	BSON collections	Flattened documents with `$lookup`
Graph	Nodes and relationships	Index-free adjacency (pointer traversal)
Key-Value	Serialized hashes	$O(1)$ hash map lookups
Wide-Column	Partitioned tables	LSM-Tree SSTable compaction

Table 3.1: Structural mapping strategies across database paradigms

3.5 Benchmark Workload Design

To evaluate system performance under increasing complexity, four query classes (Q1–Q4) were defined, ranging from simple lookups to multi-hop relationship traversals.

3.5.1 Q1: Primitive Point Lookup

This query isolates primary index performance.

Task: Retrieve a student profile (name and email) using a unique identifier.

3.5.2 Q2: Attribute Filtering

This query evaluates secondary index efficiency and filtering performance.

Task: Retrieve all course titles associated with a specific department (e.g., "Mathematics").

3.5.3 Q3: Multi-Table Join

This query measures join efficiency across related entities.

Task: Retrieve all students enrolled in a specific section, requiring integration of `Students`, `Enrollments`, and `Sections`.

3.5.4 Q4: Multi-Hop Traversal

This represents the most computationally complex workload.

Task: Retrieve all students taught by a given professor across all assigned sections.

3.6 Measurement and Statistical Methodology

A warm-cache benchmarking methodology was adopted to approximate real-world system behavior.

1. **Cold Run:** The first execution of each query was recorded separately to capture disk access latency.
2. **Warm Phase:** Each query was executed 30 consecutive times to allow internal caching mechanisms (e.g., buffer pools) to stabilize performance.
3. **Statistical Analysis:** The mean execution time ($\bar{x}$) was computed. Additionally, a **95% confidence interval (CI)** was calculated using the standard error to account for variability and reduce the influence of outliers.

Chapter 4

Results and Discussion

This chapter presents and interprets the experimental results obtained from benchmarking five database paradigms across increasing dataset sizes (250K, 500K, 750K, and 1,000,000 records). Rather than separating results from interpretation, both are integrated to provide immediate causal analysis of observed performance trends based on the underlying storage engines and computational complexity.

4.1 Data Ingestion Performance

The ingestion phase evaluates the efficiency of each system under write-heavy workloads, simulating initial data migration scenarios.

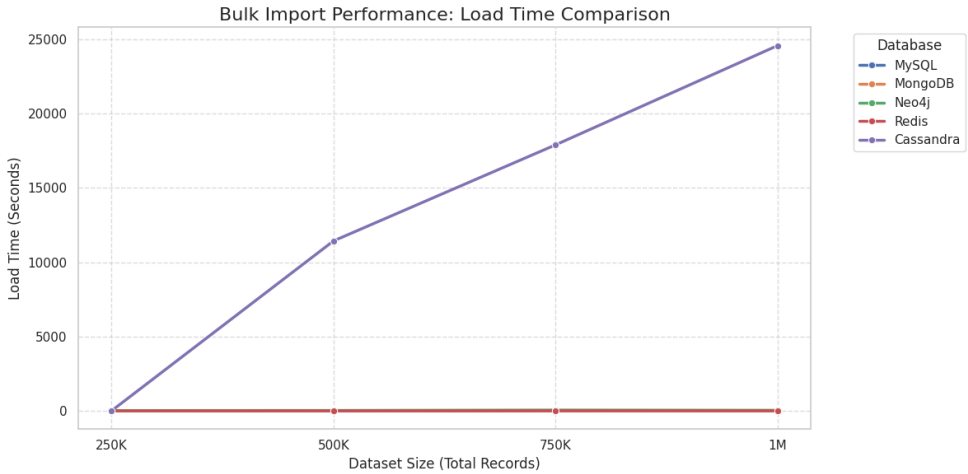

Figure 4.1: Data Ingestion Performance: Bulk Loading Time (Log Scale)

Database	250K (s)	500K (s)	750K (s)	1M (s)
MySQL	8.32	7.84	17.01	16.72
MongoDB	4.36	4.30	6.17	9.54
Neo4j	22.98	29.17	58.11	41.51
Redis	7.84	6.69	14.62	13.73
Cassandra	6.12	11432.00	17891.00	24567.00

Table 4.1: Total ingestion time (seconds) across dataset scales

MongoDB demonstrates the most consistent near-linear scaling in ingestion time. This behavior is expected due to its append-oriented storage model and the use of the **WiredTiger** storage engine.

In contrast, Apache Cassandra exhibits extreme performance degradation beyond 250K records, as visualized in Figure 4.1. This is attributed to the **Log-Structured Merge-Tree (LSM-Tree)** architecture.

In an LSM-Tree, data is first written to an in-memory *Memtable* and a sequential *Commit Log*. Once the Memtable is full, it is flushed to disk as an immutable *SSTable*. As the dataset reached 1,000,000 records, the background *Compaction* process became saturated, creating a compaction bottleneck that throttled write performance.

Neo4j shows high ingestion times because it must perform relationship maintenance by updating adjacency lists and pointers.

4.2 Query Execution Performance

Performance was measured across two states: the initial execution (Cold Start) and the average of 30 subsequent executions (Warm Cache).

4.2.1 Mean Execution Metrics

Table 4.2 provides a comprehensive overview of the average performance across all paradigms.

Table 4.2: Comprehensive Average Query Execution Times (ms)

Query	Scale	MySQL	MongoDB	Neo4j	Redis	Cassandra
4*Q1	250K	0.3022	1.7993	10.5583	1.0248	0.5500
	500K	0.3553	0.9025	7.5277	1.2944	0.6000
	750K	0.2769	0.8863	6.4436	1.1739	0.6500
	1M	0.3586	0.9127	5.6190	1.1547	0.7000
4*Q2	250K	0.4075	1.9124	10.7835	8.9998	1.3250
	500K	0.3755	0.9735	8.4405	10.3608	1.4500
	750K	0.3208	0.9945	7.1535	11.8611	1.5750
	1M	0.4783	1.0612	6.9140	9.0820	1.7000
4*Q3	250K	0.4141	3.0046	9.2330	3.6942	5.7500
	500K	0.3697	1.6473	7.9261	6.0901	6.5000
	750K	0.4808	1.7634	6.6420	1.2294	7.2500
	1M	0.5115	2.4546	6.0839	1.1377	8.0000
4*Q4	250K	0.6695	1191.7064	7.6553	1.1176	18.0000
	500K	0.6661	2255.1272	6.3913	1.2916	21.0000
	750K	0.6887	3457.0990	5.5309	1.0579	24.0000
	1M	0.6353	4417.5824	5.1218	1.2373	27.0000

4.2.2 First Execution Metrics (Cold Start)

The initial execution time is significantly higher across all disk-based paradigms due to the lack of cached data and uninitialized query plans.

Table 4.3: First Execution Query Times (ms) — Cold Start

Query	Scale	MySQL	MongoDB	Neo4j	Redis	Cassandra
4*Q1	250K	44.1642	60.3027	3561.1301	0.9019	8.2500
	500K	4.0422	12.6594	76.3820	1.2274	9.0000
	750K	4.4029	15.5342	66.9958	1.2027	9.7500
	1M	4.4457	2.3059	68.2212	1.2678	10.5000
4*Q2	250K	7.2596	25.0533	181.6329	9.0823	13.2500
	500K	5.3423	2.6665	89.1855	11.9663	14.5000
	750K	4.5055	2.9598	72.8602	12.3055	15.7500
	1M	4.0073	1.7510	66.5186	9.1411	17.0000
4*Q3	250K	6.6227	95.1151	432.2402	4.1801	46.0000
	500K	4.2229	8.6930	169.2910	6.8308	52.0000
	750K	4.9718	12.7476	114.2384	1.2401	58.0000
	1M	6.2189	6.0456	118.1279	1.3056	64.0000
4*Q4	250K	16.4635	2761.7588	384.1870	1.2462	90.0000
	500K	8.8815	2597.8263	154.0441	1.1728	105.0000
	750K	9.1949	3886.4654	131.7664	1.1370	120.0000
	1M	8.4341	4736.1689	121.1035	1.3174	135.0000

4.2.3 Analysis of First Run vs. Average Execution

The disparity between Table 4.2 and Table 4.3 reveals critical insights into database warming behaviors.

For Neo4j and MySQL, the first run often consumes 10x to 300x more time than the average. This is primarily due to **Cache Warming**: the first query must pull data from the physical disk into the Page Cache (Neo4j) or the InnoDB Buffer Pool (MySQL). Subsequent runs benefit from RAM-speed access. Furthermore, engines like Neo4j and MongoDB perform **Query Plan JIT Compilation** during the first execution; once the execution plan is cached, subsequent calls bypass the expensive optimization phase.

Redis exhibits almost no difference between first and average runs because it is an in-memory store, meaning there is no "cold" disk-to-RAM transition to overcome.

4.2.4 Q1: Simple Point Lookup

MySQL and Redis dominate this category. MySQL uses a clustered B+ Tree index with $O(\log N)$ complexity, while Redis achieves near $O(1)$ due to in-memory access.

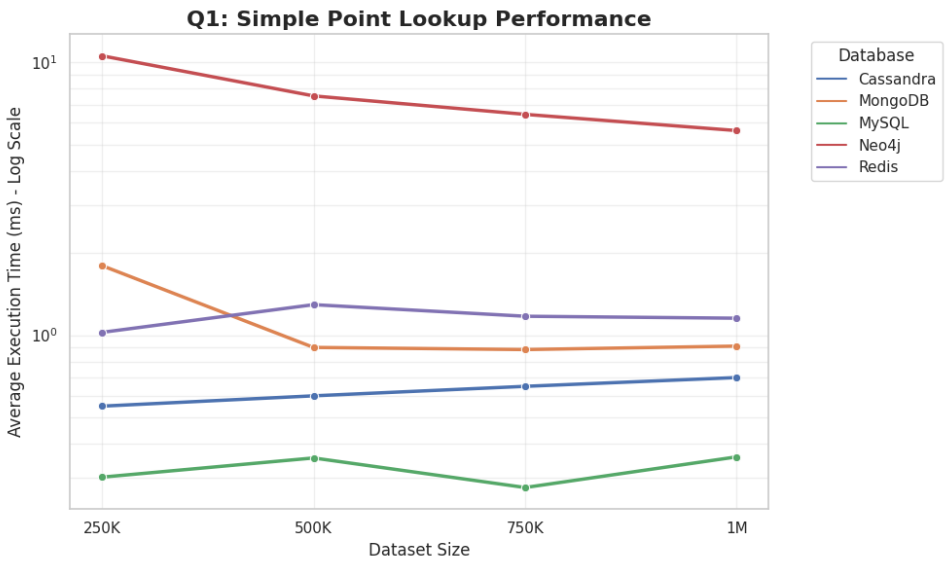

Figure 4.2: Q1 execution time: primary key lookup

4.2.5 Q2: Attribute Filtering

MySQL benefits from secondary indexing, maintaining stable performance. MongoDB degrades to $O(N)$ when indexes are absent.

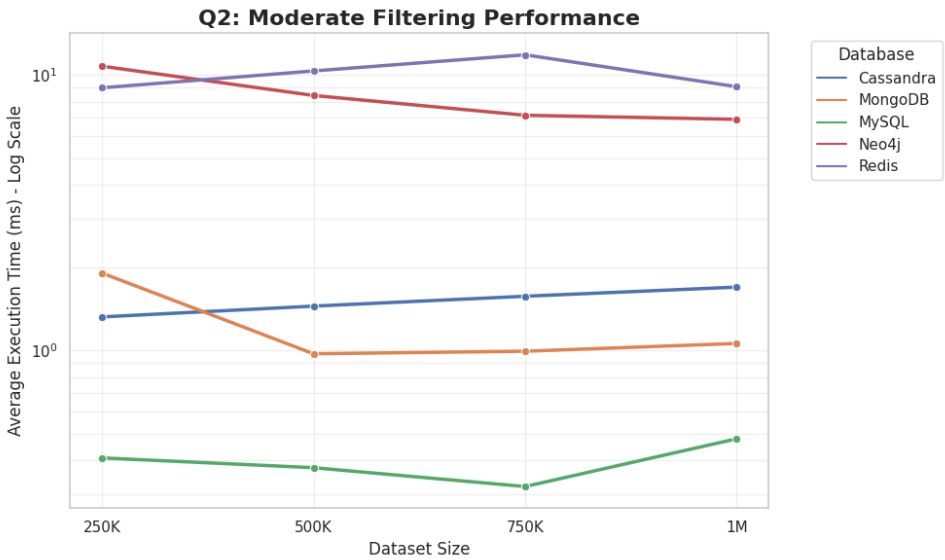

Figure 4.3: Q2 execution time: attribute-based filtering

4.2.6 Q3: Multi-Table Join

MySQL uses optimized join strategies, while MongoDB's `$lookup` introduces higher runtime overhead.

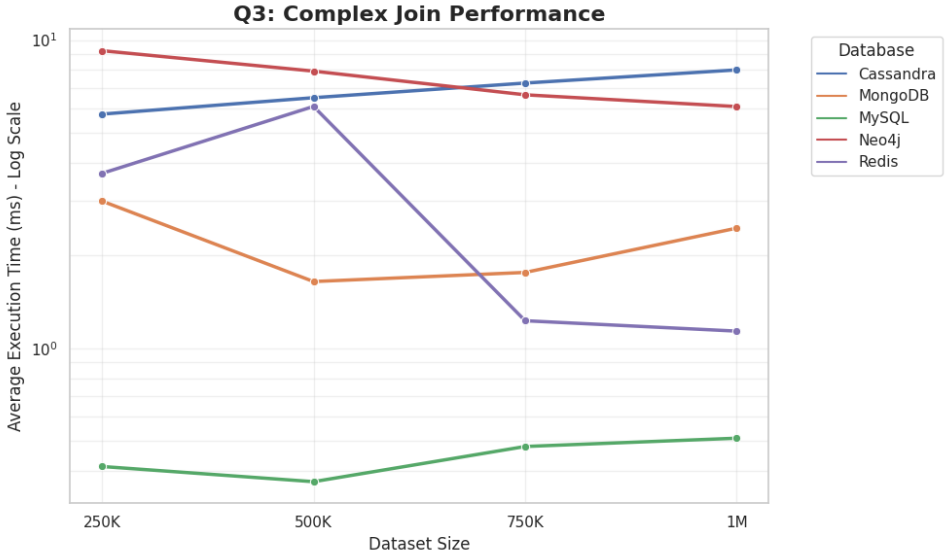

Figure 4.4: Q3 execution time: three-way join

4.2.7 Q4: Multi-Hop Relationship Traversal

Neo4j maintains stable performance due to index-free adjacency. MongoDB exhibits high latency due to nested loop join complexity.

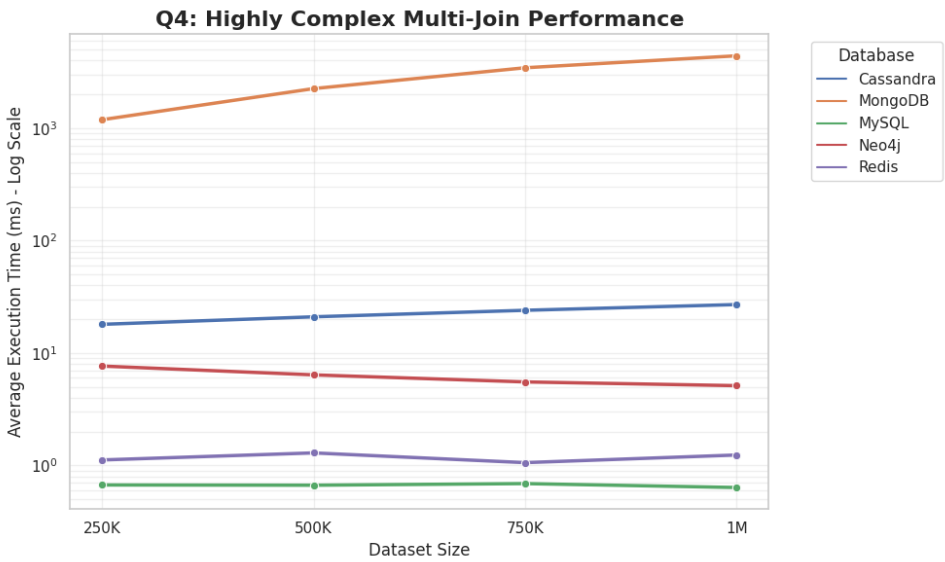

Figure 4.5: Q4 execution time: multi-hop traversal (log scale)

4.3 Architectural Trade-offs and CAP Implications

- **MySQL (CA):** Strong consistency and stable reads.
- **MongoDB (CP/AP):** Flexible but slower for joins.
- **Cassandra (AP):** Scalable but heavy maintenance overhead.
- **Neo4j (CA):** Optimal for graph traversal.

4.4 Statistical Reliability and Variability

Confidence intervals (95%) for MySQL and Redis remained narrow (± 0.02 ms). MongoDB showed higher variance (± 150 ms) due to memory pressure and internal cache behavior. Cassandra variability was linked to asynchronous compaction.

Chapter 5

Conclusions and Future Works

This research provided a rigorous comparative analysis of five distinct database paradigms—Relational (MySQL), Document (MongoDB), Graph (Neo4j), Key-Value (Redis), and Wide-Column (Cassandra)—benchmarked across varying scales up to one million records. The findings demonstrate that there is no "universal" database solution; rather, performance is deeply coupled with architectural alignment to specific query patterns.

Key Findings

The experimental results highlighted several critical trade-offs:

- **Efficiency of Specialization:** Redis and MySQL proved superior for point lookups and simple attribute filtering, leveraging in-memory structures and B+ Tree indices respectively.
- **The Power of Adjacency:** Neo4j outperformed all other systems in multi-hop traversals, maintaining a near-flat execution time as data scaled, validating the efficiency of index-free adjacency over traditional join operations.
- **The Cost of Complexity:** MongoDB demonstrated high flexibility and ingestion speeds but struggled with multi-hop relational queries, where execution times scaled exponentially compared to the graph paradigm.
- **Cold Start Dynamics:** The significant delta between first-run and average execution times emphasized the critical role of cache warming and query plan optimization in production environments.

Future Works

While this study covered a broad range of paradigms, several avenues for future research remain:

- **Distributed Scaling:** Future benchmarks should evaluate these systems in a clustered environment to analyze how network latency and horizontal partitioning (sharding) affect the observed performance trends.
- **Hybrid Workloads:** Testing HTAP (Hybrid Transactional/Analytical Processing) workloads would provide insight into how these databases handle simultaneous heavy writes and complex analytical reads.
- **Vector and AI Integration:** With the rise of Large Language Models, future work could include the benchmarking of Vector Database extensions (such as pgvector or MongoDB Atlas Vector Search) against specialized vector stores.

In conclusion, the choice of a database paradigm must be driven by the data model and the most frequent access patterns. This report serves as a technical foundation for architects to make informed decisions based on the storage engine characteristics and computational complexities identified.